the original session token to access the victim’s account.

Browsers will prevent a site hosted on a different origin from setting cookies for your API, but there are still ways that session fixation attacks can be exploited. Firstly, if the attacker can exploit an XSS attack on your domain, or any sub-domain, then they can use this to set a cookie. Secondly, Java servlet containers, which Spark uses under the hood, support different ways to store the session token on the client. The default, and safest, mechanism is to store the token in a cookie. But you can also configure the servlet container to store the session by re-writing URLs produced by the site to include the session token in the URL itself. Such URLs look like the following:

```
https://api.example.com/users/jim;JSESSIONID=l8Kjd…
```

The `;JSESSIONID=…` bit is added by the container and is parsed out of the URL on subsequent requests. This style of session storage makes it much easier for an attacker to carry out a session fixation attack because they can simply lure the user to click on a link like the following:

```
https://api.example.com/login;JSESSIONID=<attacker-controlled-session>
```

If you use a servlet container for session management, you should ensure that the session tracking-mode is set to COOKIE in your web.xml, as in the following example: